

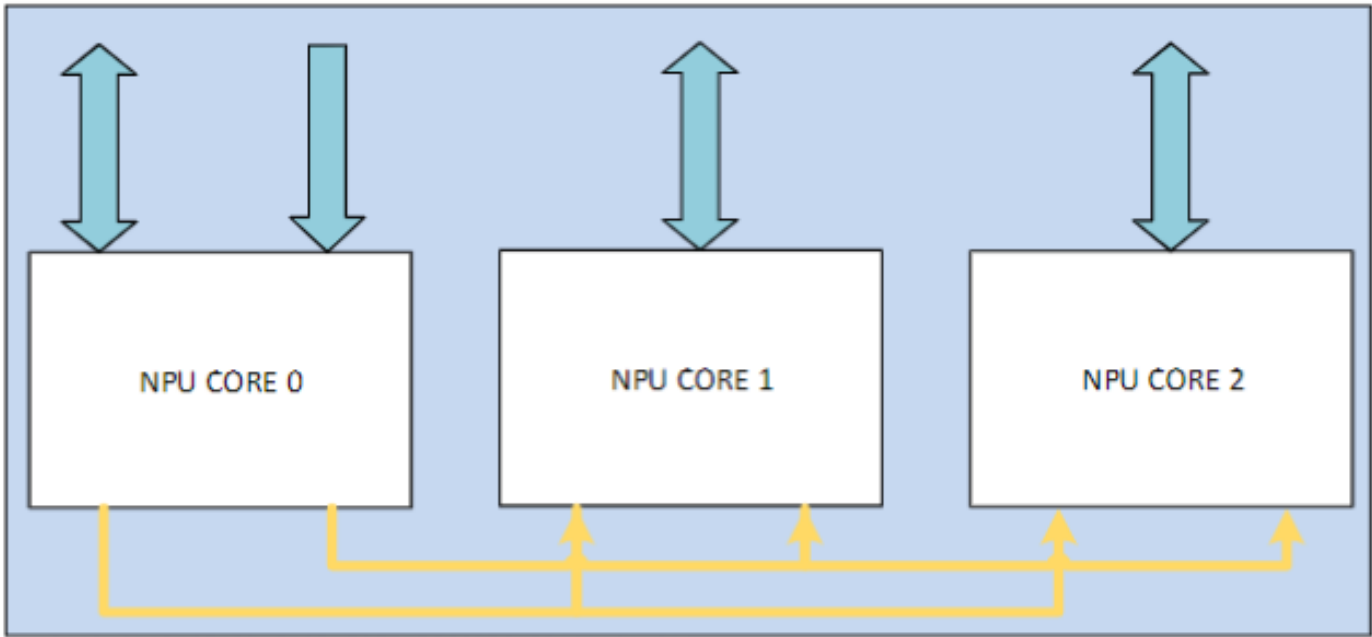
3.RKNPU推理软件架构与模型部署流程

1. 推理软件架构

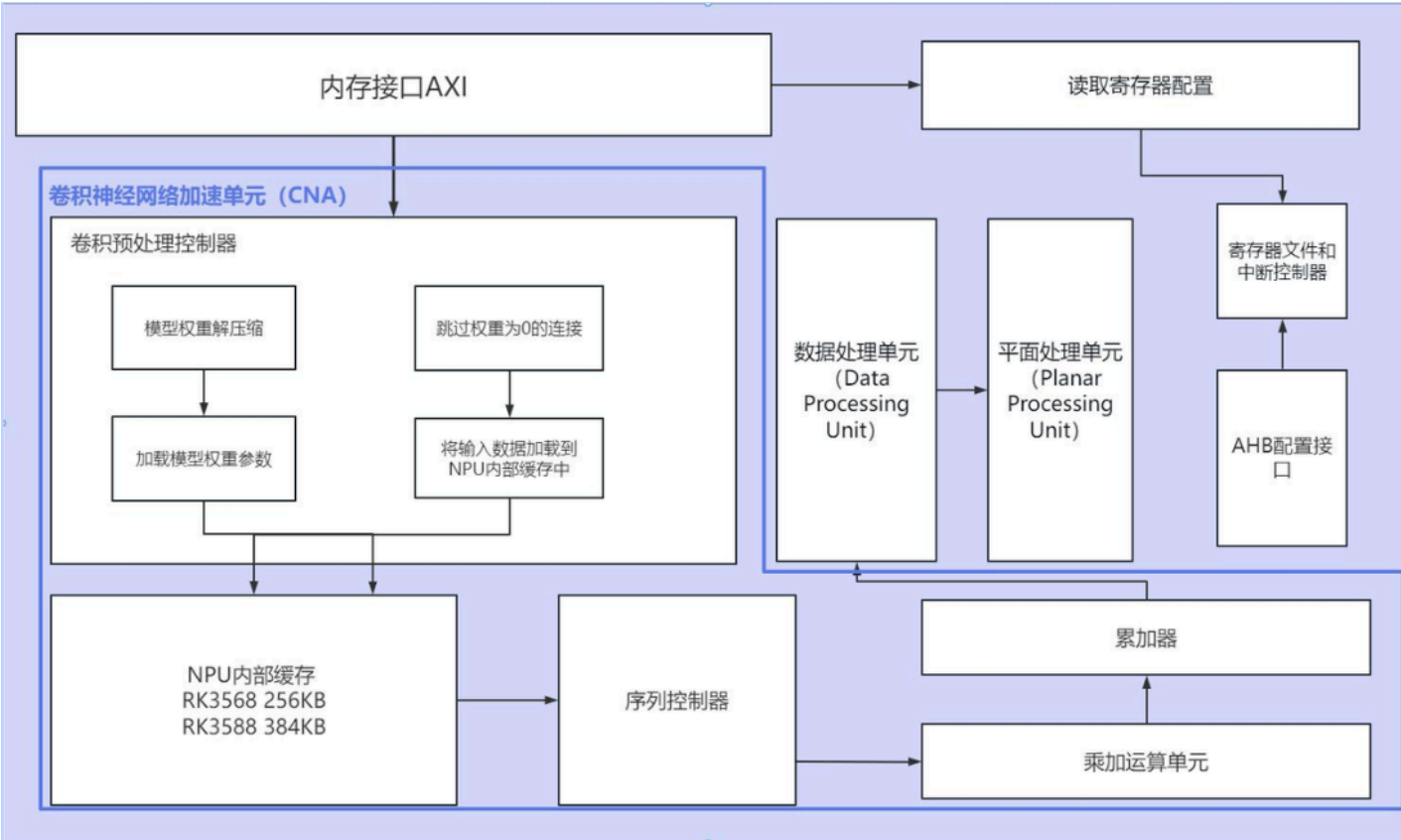


(1) RKNPU 硬件层

RKNN三核架构



RKNN单核架构



AHB/AXI 接口

AXI 主接口（AXI Master Interface）用于从连接到 SoC AXI 互联的内存中获取数据。AXI 是一种高性能、低延迟、可扩展的总线接口，常用于连接处理器和外设，并支持多个主设备和从设备。AXI 主接口通常用于从内存中获取数据，例如从 DRAM 或其他存储器中读取程序和数据，并将其传输到处理器或其他外设中进行处理和计算。

AHB 从接口（AHB Slave Interface）用于访问寄存器进行配置、调试和测试。AHB 是一种标准化的系统总线接口，通常用于连接处理器、内存和外设等硬件电路。AHB 从接口通常用于访问寄存器，例如控制和配置处理器、外设和其他硬件电路的参数和状态，以实现系统的配置、调试和测试。

在 SoC 系统中，AXI 主接口和 AHB 从接口通常被用于连接处理器、内存、外设和其他硬件电路，以实现数据传输、控制和配置等功能。AXI 接口通常用于高速数据传输和处理，而 AHB接口通常用于配置、调试和测试等低速控制操作。

卷积神经网络加速单元（CNA）

卷积神经网络加速单元（Convolutional Neural Network Accelerator，CNA）是 RKNPU 中重要的组成部分之一。包括卷积预处理控制器、NPU 内部缓存区、序列控制器、乘加运算单元和累加器，下面对各个部分进行介绍：

(1) 卷积预处理控制器

卷积预处理控制器是 CNA 中用于预处理卷积计算的硬件单元，可以对输入模型权重进行解压缩之后加载进 NPU 的内部缓冲区，并且可以判断零节点加速运算速度，最后将要推理的数据加载进 NPU 的内部缓冲区中。

(2) NPU 内部缓存区 (Internal Buffer)

NPU 内部缓存区是 CNA 中用于存储中间计算结果的缓存区。它可以高效地存储和管理卷积神经网络中的各种数据，包括输入数据、卷积核、卷积结果等。NPU 内部缓存区采用了多级缓存和数据重用技术，可以高效地利用计算资源和存储资源，从而进一步提高计算速度和效率。

(3) 序列控制器 (Sequence Controller)

序列控制器是 CNA 中用于控制卷积计算序列的硬件单元。它可以根据卷积神经网络的结构和参数，自动地配置和控制 CNA 中各个硬件单元的工作模式和参数。序列控制器还可以实现卷积计算的并行化和流水化，从而提高计算速度和效率。

(4) 乘加运算单元 (Multiply-Accumulate Unit, MAC)

乘加运算单元是 CNA 中用于执行卷积计算的硬件单元。它可以对输入数据和卷积核进行乘法和累加运算，从而得到卷积计算结果。乘加运算单元采用了高度并行的设计，可以同时执行多个卷积计算操作，从而大大提高计算速度和效率。

(5) 累加器 (Accumulator)

累加器是 CNA 中用于累加卷积计算结果的硬件单元。它可以高效地累加卷积计算结果，从而得到最终的输出结果。累加器可以采用多种精度，可以适应不同的计算精度要求。

数据处理单元 (Data Processing Unit)

数据处理单元 (Data Processing Unit) 主要处理单个数据的计算，例如 Leaky ReLU、ReLU、ReluX、Sigmoid、Tanh 等。它还提供了一些功能，例如 Softmax、转置、数据格式转换等。

数据处理单元是一种硬件电路，用于加速神经网络的计算过程。它通常被用于处理前向计算过程中的单个数据，例如卷积层和全连接层中的激活函数计算。不同的激活函数需要不同的计算操作，例如 ReLU 需要计算 $\max(0, x)$ ，Sigmoid 需要计算 $1/(1+\exp(-x))$ ，而数据处理单元可以通过硬件电路来实现这些计算操作，从而提高神经网络的计算性能和效率。

除了激活函数计算之外，数据处理单元还提供了一些其他的函数，例如 Softmax、转置、数据格式转换等。这些函数通常被用于神经网络模型的构建和优化过程中，例如将模型的输出转换为概率分布、重新排列张量的维度、将数据从一种格式转换为另一种格式等。

平面处理单元 (Planar Processing Unit)

平面处理单元 (Planar Processing Unit) 主要提供对数据处理单元的输出数据进行平面操作的功能，例如平均池化、最大值池化、最小值池化等。

平面处理单元是一种硬件电路，用于加速神经网络的计算过程。它通常被用于对数据处理单元的输出数据进行平面操作，例如在卷积神经网络中，对卷积层的输出进行池化操作，以降低数据维度和减少计算量。平面处理单元可以通过硬件电路来实现这些操作，从而提高神经网络的计算性能和效率。

平面处理单元支持多种平面操作，例如平均池化、最大值池化、最小值池化等。这些操作可以通过不同的参数来控制池化窗口的大小和步幅等，从而适应不同的应用场景和要求。

(2) RKNPU 驱动层

RKNPU 的驱动层是连接上层应用和 RKNPU 硬件的桥梁。驱动层的主要作用是将应用程序需要推理的内容提交给 RKNPU 进行计算，从而加速神经网络的训练和推理过程。具体来说，驱动层需要完成以下任务：

硬件初始化：驱动层需要初始化 RKNPU 硬件，包括设置寄存器、分配内存等操作，以确保 RKNPU 可以正常工作。

数据传输：驱动层需要将数据从主机内存传输到 RKNPU 内存中，以便进行计算。在计算完成后，驱动层还需要将计算结果传输回主机内存。

计算任务调度：驱动层需要根据应用程序的需求，管理和分配 RKNPU 的计算资源，以确保多个计算任务之间不会互相干扰。

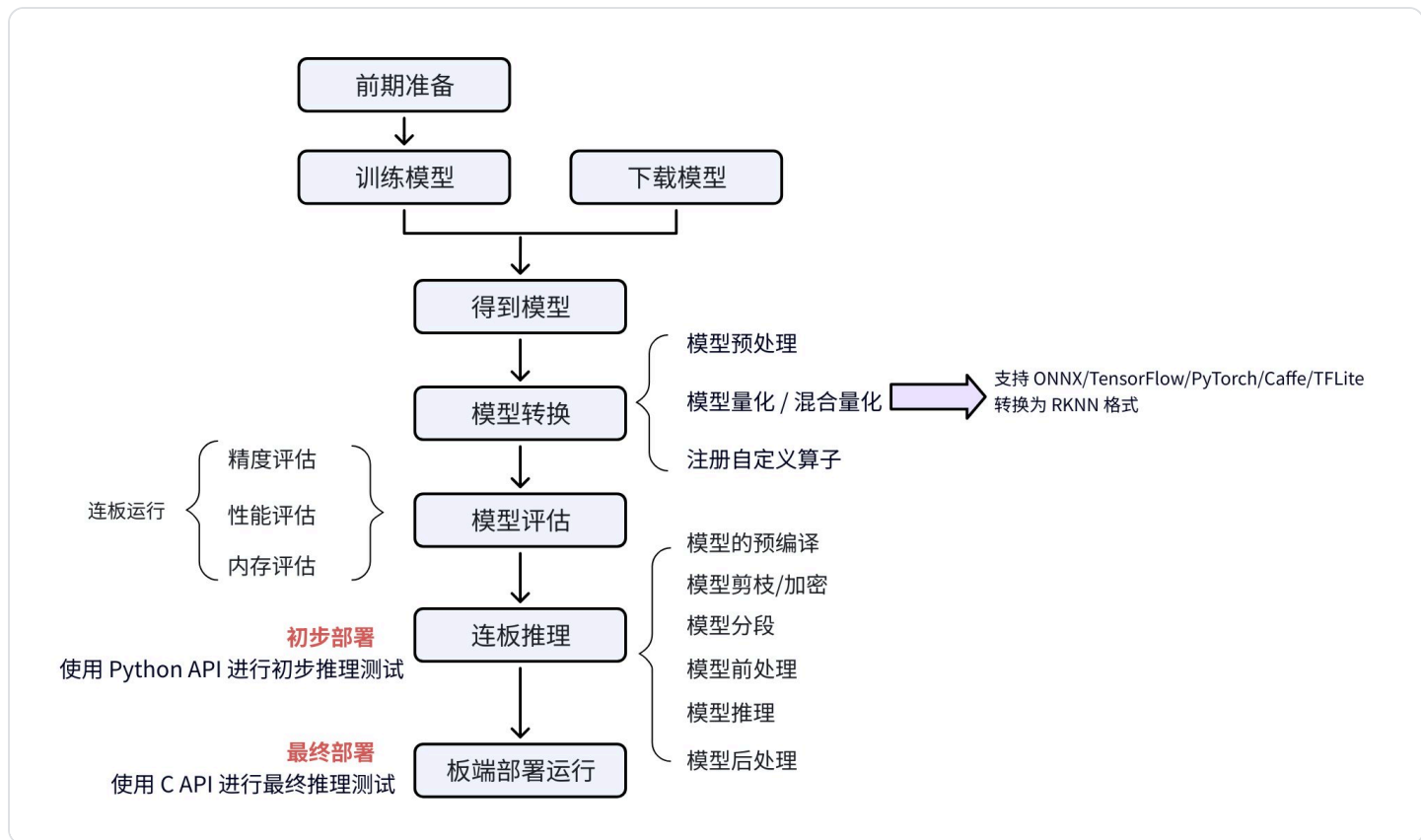
(3) RKNPU 应用层

RKNPU 应用层由 **AI 应用程序、RKNN API 以及运行时所需要的库**所组成。

开发者通过调用瑞芯微提供好的 API 接口进行 AI 应用的开发，瑞芯微分别提供了 C 语言和 Python 语言这两种 API 帮助开发者进行嵌入式设备部署，Python 语言提供的接口较为简单，旨在帮助用户进行前期的模型检测、测试以及应用调试，而要想得到更好的效果从而真正应用到实际项目中就要使用 C API 接口了。

而无论是由 C API 接口还是 Python 的 API 接口编写的应用程序，要想实际运行都需要相应的动态库，动态库包含这些 API 的具体实现，这些动态库由瑞芯微所提供。而我们只需要根据瑞芯微所提供的 API 编写对应的应用程序即可。

2. AI模型部署流程



RKNN (Rockchip Neural Network) 是瑞芯微公司开发的一种神经网络模型格式，它可以将常见的深度学习模型转换为适用于瑞芯微的 AI 加速器 RKNPU 的模型格式。RKNN 模型的优点是在保证精度的同时，可以实现高效的推理和低功耗的计算。下面是 RKNN 模型的详细介绍：

RKNN 模型的文件格式： RKNN 模型使用自定义的文件格式来表示神经网络模型。它将神经网络模型划分为两个部分：静态部分和动态部分。静态部分包括模型的网络结构和权重，而动态部分包括输入输出的形状和类型等信息。使用这种格式可以减少模型的存储空间和加载时间。

RKNN 模型的转换工具： 为了将其他常见的深度学习模型转换为 RKNN 模型，瑞芯微提供了一个转换工具 RKNN-Toolkit2。该工具支持将 TensorFlow、Caffe、MXNet 等框架训练出来的模型转换为 RKNN 模型，并且支持对模型进行量化、融合等优化操作，以提高运行效率。

RKNN 模型的部署和推理： RKNN 模型可以通过 RKNPU 硬件进行高效的推理。在部署 RKNN 模型时，开发者需要使用 RKNPU 提供的 SDK，并调用相应的 API 接口来加载和运行 RKNN 模型。由于 RKNPU 硬件的优化，RKNN 模型可以实现高效的推理和低功耗的计算。



3. rknn开发流程介绍

用户可参考如下流程图了解RKNN的整体开发步骤，流程主要分为3个部分：模型转换、模型评估和板端部署运行。



1.模型转换：

在这一阶段，原始的深度学习模型会被转化为RKNN格式，以便在RKNPu平台上进行高效的推理。这一阶段包括以下5个步骤：

- 获取原始模型：获取或训练深度学习模型，建议使用主流框架，如ONNX、PyTorch或TensorFlow。
- 模型配置：在RKNN-Toolkit2中进行必要的配置，如归一化参数、量化参数和目标平台等。
- 模型加载：使用适当的加载接口将模型导入RKNN-Toolkit2，根据模型框架选择正确的加载接口。

d. 模型构建：通过 `rknn.build()` 接口构建RKNN模型，可选择是否进行量化，提高模型在硬件上推理的性能。

e. 模型导出：通过 `rknn.export_rknn()` 接口将RKNN模型导出成一个文件（.rknn 格式），用于后续部署。

2.模型评估：

该阶段的主要目标是评估模型推理结果的准确性、推理性能和内存占用等关键指标。评估的主要步骤如下：

a. 模型连板调试：使用Python连接RKNPU平台对模型进行推理并验证结果准确性。这个阶段涵盖了输入数据的前处理和输出结果的后处理，以确保模型在板上运行正确。

b. 精度评估：通过 `rknn.accuracy_analysis()` 接口，比较量化模型与浮点模型推理结果之间的差异，以分析量化误差的来源。

c. 性能评估：通过 `rknn.eval_perf()` 接口，分析模型在RKNPU平台上的推理性能，帮助用户进一步优化模型结构，加快推理性能。

d. 内存评估：通过 `rknn.eval_memory()` 接口，了解模型在RKNPU平台上的内存使用情况，帮助用户进一步优化模型结构，最小化内存占用。

3.板端部署运行：

这个阶段涵盖了模型的实际部署和运行。它通常包括以下步骤：

a. 模型初始化：加载RKNN模型到RKNPU平台，准备进行前处理。

b. 模型前处理：加载待推理数据到RKNPU平台，准备进行推理。

c. 模型推理：执行推理操作，将输入数据传递给模型并获取推理结果。

d. 模型后处理：获取推理结果进行后处理，后处理结果传给应用端。

e. 模型释放：在完成推理流程后，释放模型资源，以便其他任务使用RKNN模型。

这三个步骤构成了完整的RKNN开发流程，确保人工智能模型能够成功转换、调试、评估和最终在RKNPU上高效部署。